

VLSI Internship Project Report on Design a pulse generator circuit that generates a single clock-cycle pulse.

Submitted by

Manish Kumar Jena (2341010080)

Mahesh Kumar Sahoo (2341019194)

Rohan Bisoi (2341002199)

Sriom Bakshi (2341002005)

Sonalika Dehury (2341004214)

B. Tech. (ECE) 7th Semester



Organized by

CAREER DEVELOPMENT CELL

Institute of Technical Education and Research

**SIKSHA 'O' ANUSANDHAN
DEEMED TO BE UNIVERSITY**

Bhubaneswar, Odisha, India.

(July 2026)

Declaration

We, the undersigned B. Tech. students from Department of (ECE), hereby declare that we own the full responsibility for the information, results etc. provided in this internship Project titled **“Design a pulse generator circuit that generates a single clock-cycle pulse”** submitted to Siksha ‘O’ Anusandhan (Deemed to be University), Bhubaneswar for the partial fulfillment of the VLSI Internship Program. We have taken care in all respect to honor the intellectual property right and have acknowledged the contribution of others for using them in academic purpose and further declare that in case of any violation of intellectual property right or copyright we, as the candidate(s), will be fully responsible for the same.

Manish Kumar Jena	Reg. No.: 231010080
Mahesh Kumar Sahoo	Reg. No.: 2341019194
Rohan Bisoi	Reg. No.: 2341002199
Sriom Bakshi	Reg. No.: 2341002005
Sonalika Dehury	Reg. No.: 2341004214

Date: 11-08-2026

Place: Bhubaneswar

REPORT APPROVAL

This project report titled “**Design of a pulse generator circuit that generates a single clock-cycle pulse**” submitted by **Mahesh Kumar Sahoo, Rohan Bisoi, Sonalika Dehury, Sriom Baksi, Manis Kumar Jena**, of **Branch ECE** to the Institute of Technical Education and Research, Siksha ‘O’ Anusandhan (Deemed to be) University, Bhubaneswar for the partial fulfillment of the VLSI Internship Program conducted by Career Development Cell is approved.

(Name and signature of the Examiner)

(Name and signature of the Internship-in-Charge)

(Name and signature of the Head of Career Development Cell)

Contents

Sl.No.	Title	Page No.
1	Introduction	1-2
2	Block Diagram	2-3
3	Modules & Codes	5-8
4	Testbench Code	4-8
5	Wave Forms	8-9
6	Synthesis Scripts & Reports	10-14
7	Static Timing Analysis Reports	15-16
8	Conclusion	17

1. Introduction

This project focuses on the design and verification of a digital pulse generator, specifically functioning as a synchronous rising-edge detector. The primary objective of this circuit is to monitor an input signal and generate a single clock-cycle pulse exactly when the input transitions from a logic low (0) to a logic high (1). In digital system design, this type of circuit is highly valuable for synchronizing asynchronous events or external inputs with a system clock. Common applications include capturing button presses without generating multiple triggers, initiating state machine transitions, acting as a clock enable for downstream registers, and generating single-cycle interrupts for microcontrollers. While the problem statement outlines a requirement for Static Timing Analysis (STA) to verify setup and hold timing constraints, the provided implementation focuses purely on the RTL design and its functional verification.

DUT Working and Functionality

The Design Under Test (DUT), named project, is a synchronous module driven by a system clock (clk) and an active-high reset (rst). The module operates using a sequential always block that evaluates on the positive edge of the clock or the reset signal. To detect a transition, the circuit must retain memory of the signal's previous state; this is achieved using an internal register named pre_in.

Reset State: When the rst signal is asserted, the output pulse_out is immediately driven to 0 to ensure a known initial state. Simultaneously, pre_in is assigned the current value of signal_in.

Active State: When the reset is de-asserted, the circuit evaluates the input on every clock cycle. The edge-detection logic uses the bitwise operation $\text{signal_in} \ \& \ (\sim \text{pre_in})$. This ensures that pulse_out is asserted high only if the current input is 1 and the previous input was 0. During this same cycle, pre_in is updated with the current signal_in value to prepare for the subsequent clock cycle's evaluation. If the input remains high across multiple cycles, pre_in will be 1, causing $(\sim \text{pre_in})$ to evaluate to 0, thereby keeping the output low and ensuring only a single-cycle pulse is generated.

UVM Verification Methodology

The design is rigorously tested using the Universal Verification Methodology (UVM) to ensure functional correctness under random and specific edge-case scenarios.

Interface and Transactions: Communication between the testbench and the DUT is handled via pulse_if, which utilizes clocking blocks (drv_cb and mon_cb) to prevent race conditions during signal driving and sampling. The data packets are defined in pulse_item, which contains randomized variables for the input signals.

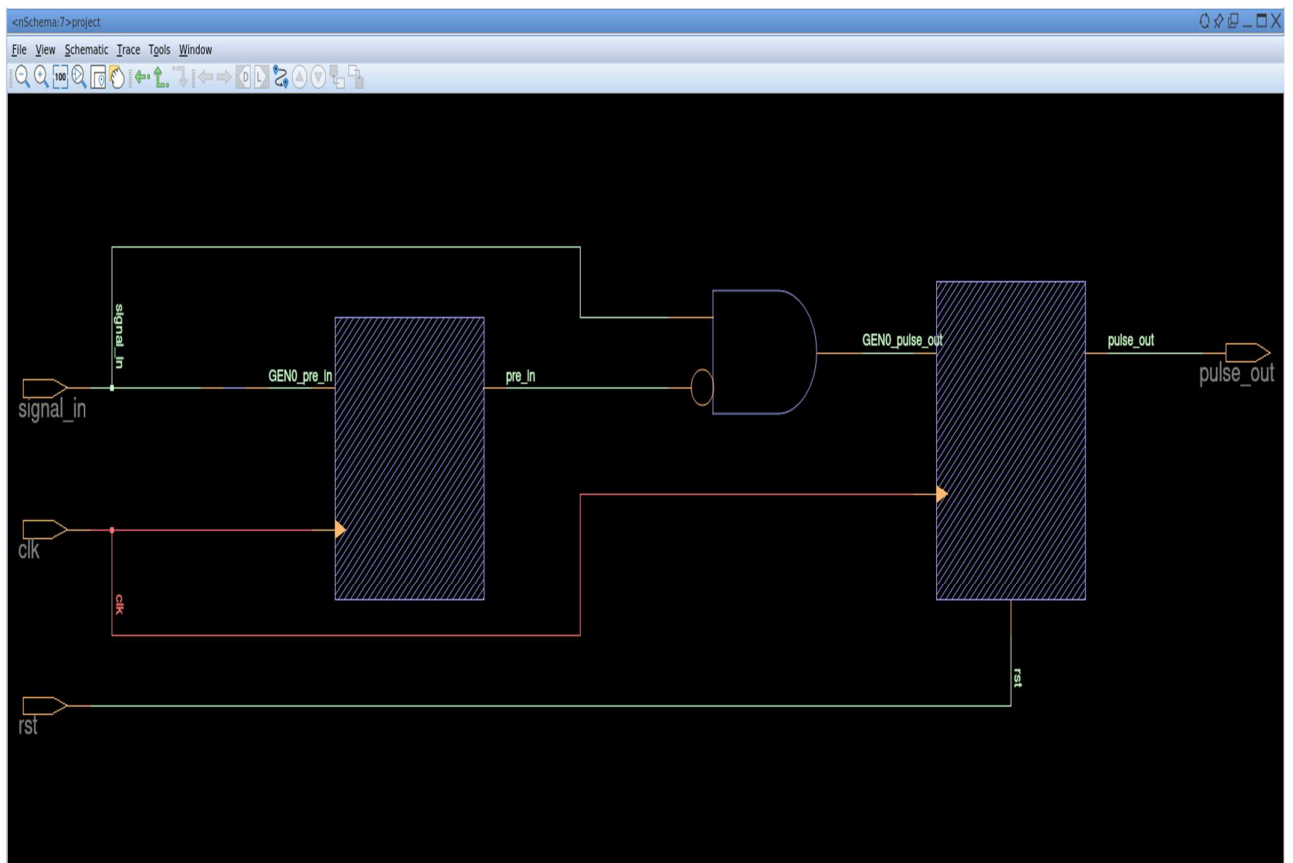
Stimulus Generation: The pulse_sequence class is responsible for stimulus generation, creating a loop of 20 randomized transactions that are fed to the driver via the sequencer (pulse_seqr). The pulse_driver then translates these transaction-level items into pin-level wiggles on the virtual interface.

Monitoring and Checking: The pulse_monitor passively observes the interface, capturing the input and output signals on every clock cycle and broadcasting them through an analysis port. These observed transactions are received by the pulse_scoreboard. The scoreboard maintains its own internal reference model (tracking prev_signal and calculating an expected_pulse) to verify the DUT's behavior. It compares the actual pulse_out from the DUT against its predicted value, logging a "PASS" or "FAIL" message accordingly.

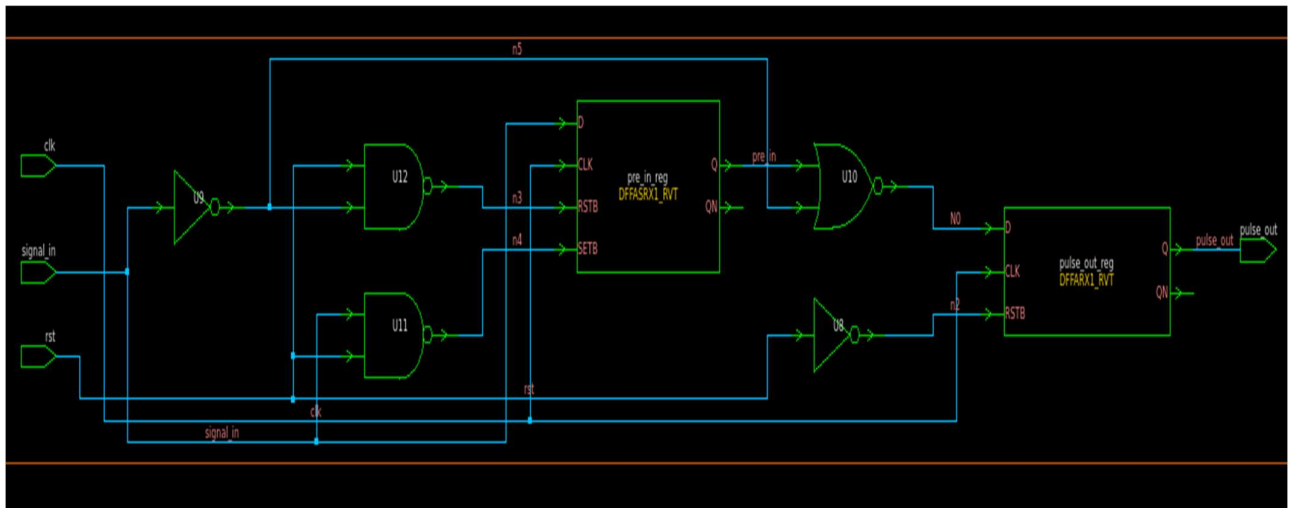
Top-Level Execution: Finally, the pulse_top module acts as the simulation harness. It generates a continuous clock with a period of 10 time units, executes an initial reset sequence, stores the virtual interface in the uvm_config_db for lower-level components to access, and initiates the dumping of waveform data to an FSDB file for debugging.

2. Block Diagram

Block Diagram From Synopsys Verdi:



Synthesized Design From Synopsys Design Compiler:



3. Modules and codes:

Design Module:

```
module project(  
    input logic clk,  
    input logic rst,  
    input logic signal_in,  
  
    output logic pulse_out);  
  
    logic pre_in; //Store previous input  
  
    always @(posedge clk or posedge rst) begin  
        if(rst) begin  
            pre_in <= signal_in;  
            pulse_out <= 1'b0;  
        end  
        else begin  
            pulse_out <= signal_in & (~pre_in);  
            pre_in <= signal_in;  
        end  
    end  
end  
endmodule
```

4. Testbench code

UVM Top Module:

```
import uvm_pkg::*;
`include "uvm_macros.svh"
`include "pulse_if.sv"
`include "pulse_item.sv"
`include "pulse_sequence.sv"
`include "pulse_seqr.sv"
`include "pulse_driver.sv"
`include "pulse_monitor.sv"
`include "pulse_agt.sv"
`include "pulse_scoreboard.sv"
`include "pulse_env.sv"
`include "pulse_test.sv"
module pulse_top;
logic clk;
initial begin
    clk=0;
    forever #5 clk=~clk;
end
pulse_if intf(clk);
project dut(.clk(intf.clk),
            .rst(intf.rst),
            .signal_in(intf.signal_in),
            .pulse_out(intf.pulse_out));
initial begin
    intf.rst = 1; #32;
    intf.rst = 0;
end
initial begin
    uvm_config_db#(virtual pulse_if)::set(null,"*", "vif", intf);
    run_test("pulse_test");
end
initial begin
    $fsdbDumpfile("pulse.fsdb");
    $fsdbDumpvars(0,pulse_top);
end
endmodule
```

Test Class:

```
class pulse_test extends uvm_test;
    `uvm_component_utils(pulse_test)
    pulse_env env;
    function new(string name="pulse_test",uvm_component parent);
        super.new(name,parent);
    endfunction
    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        env=pulse_env::type_id::create("env",this);
    endfunction
    task run_phase(uvm_phase phase);
        pulse_sequence seq;
        phase.raise_objection(this);
        seq=pulse_sequence::type_id::create("seq");
        seq.start(env.agent.seqr);
        #20;
        phase.drop_objection(this);
    endtask
endclass
```


Environment Class:

```
class pulse_env extends uvm_env;
  `uvm_component_utils(pulse_env)
  pulse_agt agent;
  pulse_scoreboard scoreboard;
  function new(string name="pulse_env",uvm_component parent);
    super.new(name,parent);
  endfunction
  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    agent=pulse_agt::type_id::create("agent",this);
    scoreboard=pulse_scoreboard::type_id::create("scoreboard",this)
;
  endfunction
  function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);
    agent.monitor.ap.connect(scoreboard.analysis_export);
  endfunction
endclass
```

Agent Class:

```
class pulse_agt extends uvm_agent;
  `uvm_component_utils(pulse_agt)
  pulse_driver driver;
  pulse_monitor monitor;
  pulse_seqr seqr;

  function new(string name="pulse_agt",uvm_component parent);
    super.new(name,parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    driver=pulse_driver::type_id::create("driver",this);
    monitor=pulse_monitor::type_id::create("monitor",this);
    seqr=pulse_seqr::type_id::create("seqr",this);
  endfunction

  function void connect_phase(uvm_phase phase);
    driver.seq_item_port.connect(seqr.seq_item_export);
  endfunction
endclass
```

Sequence Class:

```
class pulse_sequence extends uvm_sequence#(pulse_item);
  `uvm_object_utils(pulse_sequence)

  function new(string name="pulse_sequence");
    super.new(name);
  endfunction

  task body();
    pulse_item req;
    repeat(20) begin
      req=pulse_item::type_id::create("req");
      start_item(req);
      assert(req.randomize());
      finish_item(req);
    end
  endtask
endclass
```

Sequencer Class:

```
class pulse_seqr extends uvm_sequencer#(pulse_item);
    `uvm_component_utils(pulse_seqr)

    function new(string name="pulse_seqr",uvm_component parent);
        super.new(name,parent);
    endfunction
endclass
```

Driver Class:

```
class pulse_driver extends uvm_driver#(pulse_item);
    `uvm_component_utils(pulse_driver)

    virtual pulse_if vif;

    function new(string name="pulse_driver",uvm_component parent);
        super.new(name,parent);
    endfunction
    function void build_phase(uvm_phase phase);
        super.build_phase(phase);

    if(!uvm_config_db#(virtualpulse_if)::get(this,"","vif",vif))
        `uvm_fatal("DRV","Interface not found");
    endfunction
    task run_phase(uvm_phase phase);
        pulse_item req;
        forever begin
            seq_item_port.get_next_item(req);
            @(posedge vif.clk);
            vif.signal_in <= req.signal_in;

            `uvm_info("DRIVER","Drivertransaction:",UVM_MEDIUM)
            req.print();
            seq_item_port.item_done();
        end
    endtask
endclass
```

Monitor Class:

```
class pulse_monitor extends uvm_monitor;
    `uvm_component_utils(pulse_monitor)

    virtual pulse_if vif;
    uvm_analysis_port#(pulse_item)ap;

    function new(string name="pulse_monitor",uvm_component parent);
        super.new(name,parent);
        ap=new("ap",this);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        if(!uvm_config_db#(virtual
pulse_if)::get(this,"","vif",vif))
            `uvm_fatal("MON","Interface not found");
        endfunction

    task run_phase(uvm_phase phase);
        pulse_item item;
        forever begin
            @(vif.mon_cb);
```

```

        item=pulse_item::type_id::create("item");
        item.rst=vif.mon_cb.rst;
        item.signal_in=vif.mon_cb.signal_in;
        item.pulse_out=vif.mon_cb.pulse_out;
        `uvm_info("MONITOR","Received
transaction:",UVM_MEDIUM)
        item.print();
        ap.write(item);
    end
endtask
endclass

```

Interface:

```

interface pulse_if(input logic clk);
    logic rst;
    logic signal_in;
    logic pulse_out;

    clocking drv_cb @ (posedge clk);
        default input #1step output #1step;
        output rst;
        output signal_in;

        input pulse_out;
    endclocking

    clocking mon_cb @ (posedge clk);
        default input #1step;
        input rst;
        input signal_in;
        input pulse_out;
    endclocking

    modport DRIVER(clocking drv_cb,input clk);
    modport MONITOR(clocking mon_cb,input clk);

endinterface

```

Item Class:

```

class pulse_item extends uvm_sequence_item;
    rand bit signal_in;
    bit rst;

    bit pulse_out;

    `uvm_object_utils_begin(pulse_item)
    `uvm_field_int(rst,UVM_ALL_ON)

    `uvm_field_int(signal_in,UVM_ALL_ON)
    `uvm_field_int(pulse_out,UVM_ALL_ON)

    `uvm_object_utils_end

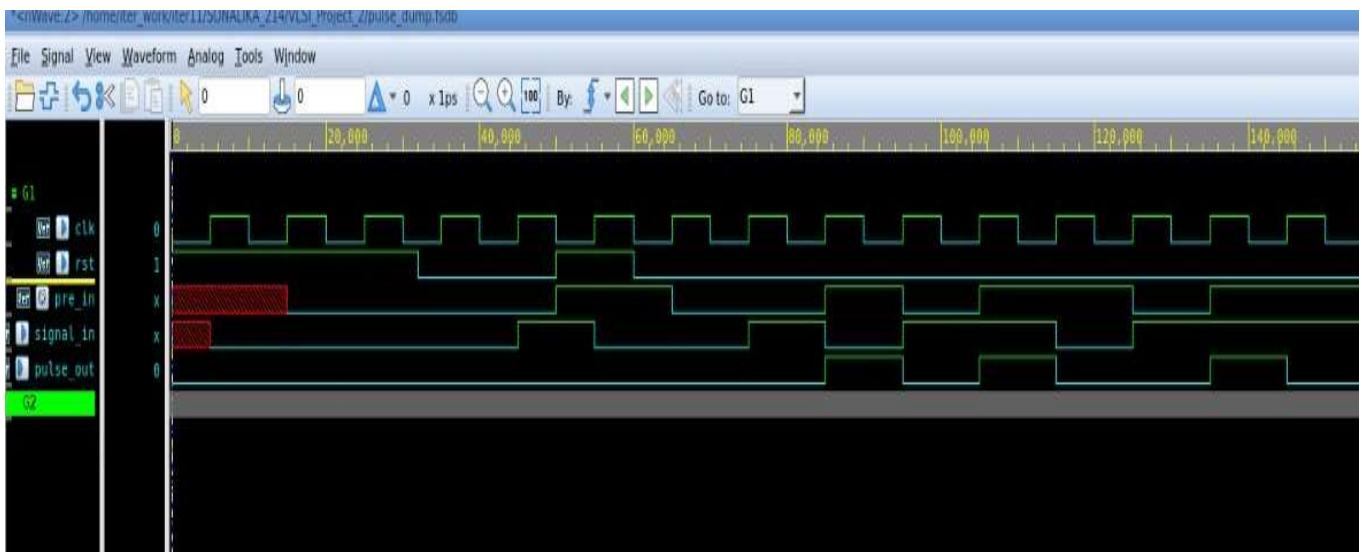
    function new(string name="pulse_item");
        super.new(name);
    endfunction
endclass

```

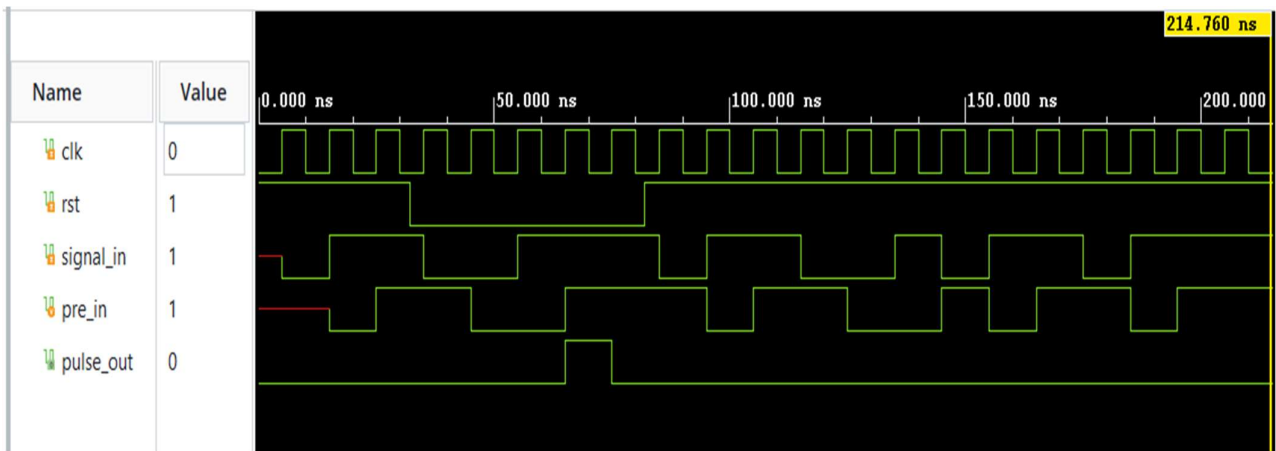
Scoreboard Class:

```
class pulse_scoreboard extends uvm_scoreboard;
  `uvm_component_utils(pulse_scoreboard)
  uvm_analysis_imp #(pulse_item, pulse_scoreboard)
  analysis_export;
  bit prev_signal;
  bit expected_pulse;
  function new(string name = "pulse_scoreboard", uvm_component
parent);
    super.new(name, parent);
    analysis_export = new("analysis_export", this);
  endfunction
  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    prev_signal = 0;
    expected_pulse = 0;
  endfunction
  virtual function void write(pulse_item tr);
  if (tr.rst) begin
    prev_signal <= tr.signal_in;
    expected_pulse <= 0;
  end
  else begin
    expected_pulse <= tr.signal_in & ~prev_signal;
    prev_signal <= tr.signal_in;
  end
  if (tr.pulse_out == expected_pulse) begin
    `uvm_info("SCOREBOARD",
      $sformatf("PASS : signal_in=%0b pulse_out=%0b",
        tr.signal_in, tr.pulse_out),
        UVM_LOW)
  end
  else begin
    `uvm_error("SCOREBOARD", $sformatf("FAIL:signal_in=%0bExpected=%0b
Actual=%0b", tr.signal_in, expected_pulse, tr.pulse_out));
  end
  endfunction
endclass
```

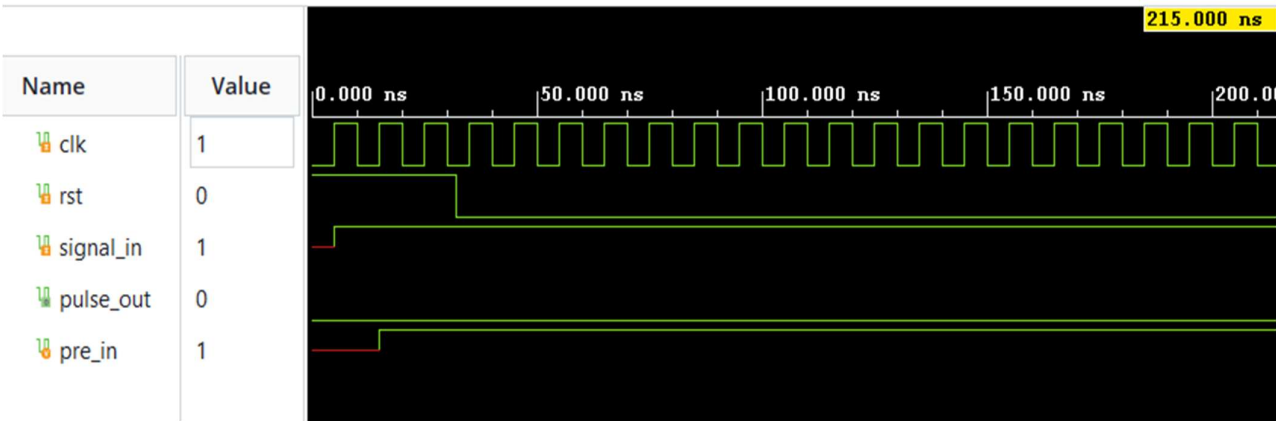
5. Wave forms



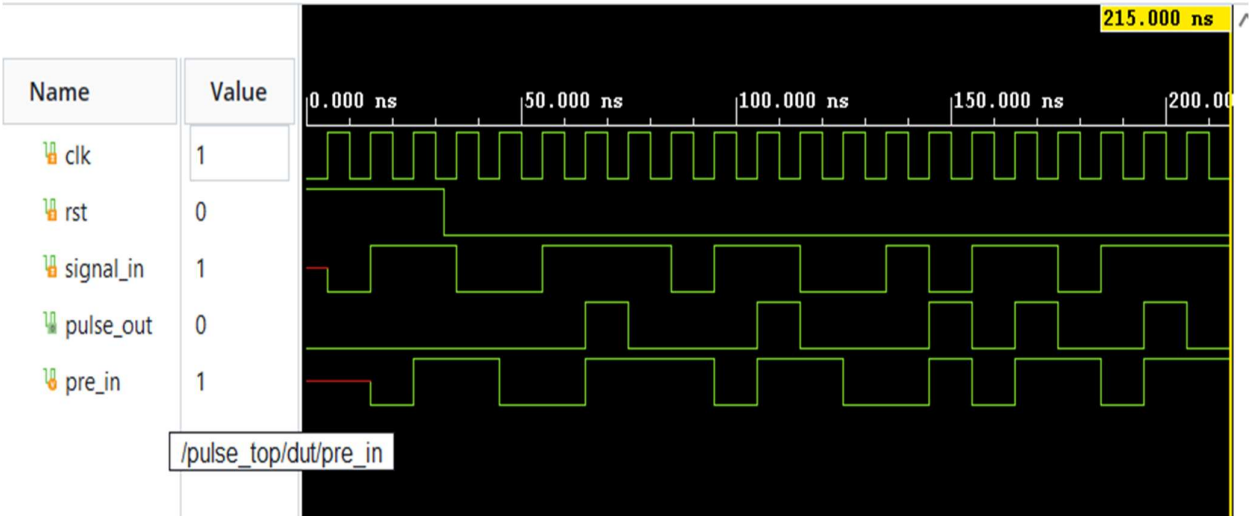
Reset Verification



Signal Remains High



Back To Back rising edge test:



6. Synthesis Scripts And Reports

Design Compiler TCL Script:

```
read_file -format ddc {/home/iter_work/iter13/Mahesh_9194/Synthesis/baudrate_work.ddc}

set link_library {* /home/iter_work/KIT/SAED32_EDK/lib/stdcell_rvt/db_nldm/saed32rvt_tt0p85v25c.db}
set target_library {/home/iter_work/KIT/SAED32_EDK/lib/stdcell_rvt/db_nldm/saed32rvt_tt0p85v25c.db}

analyze -library WORK -format verilog /home/iter_work/iter13/Mahesh_9194/UART_Synthesis/project.v
|
elaborate top

create_clock -name clk -period 10
create_clock -name clk -period 10 [get_ports clk]
set_input_delay 2.5 -clock clk [all_inputs]
set_output_delay 1 -clock clk [all_outputs]

link

set_driving_cell -lib_cell NBUFFX2_RVT [all_inputs]

compile

check_timing

report_timing
report_area
report_power
report_design
report_cell
report_qor
```

Area Report:

```
dc_shell> report_area

*****
Report : area
Design : project
Version: W-2024.09-SP3
Date   : Mon Aug 10 11:10:46 2026
*****

Library(s) Used:

    saed32rvt_tt0p85v25c (File: /home/iter_work/KIT/SAED32_EDK/lib/stdcell_rvt/db_nldm/saed32rvt_tt0p85v25c.db)

Number of ports:                4
Number of nets:                 10
Number of cells:                7
Number of combinational cells:  5
Number of sequential cells:     2
Number of macros/black boxes:   0
Number of buf/inv:              2
Number of references:           5

Combinational area:             8.132608
Buf/Inv area:                   2.541440
Noncombinational area:          14.740352
Macro/Black Box area:           0.000000
Net Interconnect area:          1.453394

Total cell area:                22.872960
Total area:                     24.326354
1
```

Power Report:

```
1
x dc_shell> report_power
Information: Propagating switching activity (low effort zero delay simulation). (PWR-6)
Warning: Design has unannotated primary inputs. (PWR-414)
Warning: Design has unannotated sequential cell outputs. (PWR-415)

*****
Report : power
        -analysis_effort low
Design : project
Version: W-2024.09-SP3
Date   : Mon Aug 10 11:10:59 2026
*****

Library(s) Used:

saed32rvt_tt0p85v25c (File: /home/iter_work/KIT/SAED32_EDK/lib/stdcell_rvt/db_nldm/saed32rvt_tt0p85v25c.db)

Operating Conditions: tt0p85v25c Library: saed32rvt_tt0p85v25c
Wire Load Model Mode: enclosed

Design      Wire Load Model      Library
-----
project      ForQA                  saed32rvt_tt0p85v25c

Global Operating Voltage = 0.85
Power-specific unit information :
  Voltage Units = 1V
  Capacitance Units = 1.000000ff
  Time Units = 1ns
  Dynamic Power Units = 1uW (derived from V,C,T units)
  Leakage Power Units = 1pW

Attributes
-----
i - Including register clock pin internal power

Cell Internal Power = 989.8463 nW (97%)
Net Switching Power = 27.3990 nW (3%)
-----
Total Dynamic Power = 1.0172 uW (100%)

Cell Leakage Power = 581.1299 nW

Power Group      Internal      Switching      Leakage      Total
                  Power        Power          Power        Power ( % ) Attrs
-----
io_pad           0.0000        0.0000        0.0000        0.0000 ( 0.00%)
memory           0.0000        0.0000        0.0000        0.0000 ( 0.00%)
black_box        0.0000        0.0000        0.0000        0.0000 ( 0.00%)
clock_network    0.9437        0.0000        0.0000        0.9437 ( 59.04%) i
register          2.0597e-02    3.4065e-03    4.8385e+05    0.5079 ( 31.77%)
sequential       0.0000        0.0000        0.0000        0.0000 ( 0.00%)
combinational    2.5582e-02    2.3993e-02    9.7280e+04    0.1469 ( 9.19%)
-----
Total            0.9898 uW     2.7399e-02 uW  5.8113e+05 pW  1.5984 uW
1
```

Cell Report:

```
Hier.1 x Schematic.2 x
1
dc_shell> get_cell
{pre_in_reg pulse_out_reg U8 U9 U10 U11 U12}
dc_shell> report_cell

*****
Report : cell
Design : project
Version: W-2024.09-SP3
Date   : Mon Aug 10 11:07:31 2026
*****

Attributes:
  b - black box (unknown)
  h - hierarchical
  n - noncombinational
  r - removable
  u - contains unmapped logic

Cell          Reference      Library          Area  Attributes
-----
U8            INVX0_RVT      saed32rvt_tt0p85v25c  1.270720
U9            INVX0_RVT      saed32rvt_tt0p85v25c  1.270720
U10           NOR2X0_RVT      saed32rvt_tt0p85v25c  2.541440
U11           NAND2X0_RVT     saed32rvt_tt0p85v25c  1.524864
U12           NAND2X0_RVT     saed32rvt_tt0p85v25c  1.524864
pre_in_reg    DFFASRX1_RVT    saed32rvt_tt0p85v25c  7.624320  n
pulse_out_reg DFFARX1_RVT     saed32rvt_tt0p85v25c  7.116032  n
-----
Total 7 cells          22.872960
1
```

Design Report:

```
Hier.1 x Schematic.2 x
dc_shell> report_design

*****
Report : design
Design : project
Version: W-2024.09-SP3
Date   : Mon Aug 10 11:10:35 2026
*****

Design allows ideal nets on clock nets.

Library(s) Used:

  saed32rvt_tt0p85v25c (File: /home/iter_work/KIT/SAED32_EDK/lib/stdcell_rvt/db_nldm/saed32rvt_tt0p85v25c.db)

Local Link Library:

  {/home/iter_work/KIT/SAED32_EDK/lib/stdcell_rvt/db_nldm/saed32rvt_tt0p85v25c.db}

Flip-Flop Types:

  No flip-flop types specified.

Latch Types:

  No latch types specified.

Operating Conditions:

  Operating Condition Name : tt0p85v25c
  Library : saed32rvt_tt0p85v25c
  Process : 1.00
  Temperature : 25.00
  Voltage : 0.85
  Interconnect Model : balanced_tree
```


Wire Loading Model:					
Selected automatically from the total cell area.					
Name	:	ForQA			
Location	:	saed32rvvt_tt0p85v25c			
Resistance	:	0.002067			
Capacitance	:	0.026724			
Area	:	0.01			
Slope	:	30.2854			
Fanout	Length	Points	Average Cap	Std Deviation	
1	8.28				
2	18.49				
3	29.35				
4	40.92				
5	53.23				
6	66.36				
7	80.36				
8	95.27				
9	111.17				
10	128.09				
11	146.10				
12	165.26				
13	185.61				
14	207.22				
15	230.13				
16	254.41				
17	280.11				
18	307.28				
19	335.98				
20	366.27				

Wire Loading Model Mode: enclosed.

Timing Ranges:

No timing ranges specified.

Pin Input Delays:

None specified.

Pin Output Delays:

None specified.

Disabled Timing Arcs:

No arcs disabled.

Required Licenses:

None Required

Design Parameters:

None specified.

1

Quality Of Reports / QOR Report:

```
dc_shell> report_qor

*****
Report : qor
Design : project
Version: W-2024.09-SP3
Date   : Mon Aug 10 11:22:26 2026
*****

Timing Path Group 'clk'
-----
Levels of Logic:          2.00
Critical Path Length:     0.15
Critical Path Slack:      7.79
Critical Path Clk Period: 10.00
Total Negative Slack:     0.00
No. of Violating Paths:   0.00
Worst Hold Violation:     0.00
Total Hold Violation:     0.00
No. of Hold Violations:   0.00
-----

Cell Count
-----
Hierarchical Cell Count:  0
Hierarchical Port Count:  0
Leaf Cell Count:         7
Buf/Inv Cell Count:      2
Buf Cell Count:          0
Inv Cell Count:          2
CT Buf/Inv Cell Count:   0
Combinational Cell Count: 5
Sequential Cell Count:   2
Macro Count:             0
-----

Area
-----
Combinational Area:      8.132608
Noncombinational Area:   14.740352
Buf/Inv Area:            2.541440
Total Buffer Area:       0.00
Total Inverter Area:     2.54
Macro/Black Box Area:    0.000000
Net Area:                1.453394
-----
Cell Area:               22.872960
Design Area:             24.326354

Design Rules
-----
Total Number of Nets:    10
Nets With Violations:    0
Max Trans Violations:    0
Max Cap Violations:      0
-----

Hostname: taradash.soa.ac.in

Compile CPU Statistics
-----
Resource Sharing:         0.00
Logic Optimization:       0.06
Mapping Optimization:     0.34
-----
Overall Compile Time:     6.15
Overall Compile Wall Clock Time: 6.56
-----

Design WNS: 0.00 TNS: 0.00 Number of Violating Paths: 0

Design (Hold) WNS: 0.00 TNS: 0.00 Number of Violating Paths: 0
-----
```

7. Static Timing Analysis reports

Setup Time Report:

```
dc_shell> report_timing -delay_type max
```

Report : timing

-path full

-delay max

-max_paths 1

Design : project

Version: W-2024.09-SP3

Date : Sat Aug 8 12:47:26 2026

Operating Conditions: tt0p85v25c Library: saed32rvt_tt0p85v25c

Wire Load Model Mode: enclosed

Startpoint: signal_in (input port clocked by clk)

Endpoint: pulse_out_reg

(rising edge-triggered flip-flop clocked by clk)

Path Group: clk

Path Type: max

Des/Clust/Port	Wire Load Model	Library	

project	ForQA	saed32rvt_tt0p85v25c	
Point	Incr	Path	

clock clk (rise edge)	0.00	0.00	
clock network delay (ideal)	0.00	0.00	
input external delay	2.00	2.00 r	
signal_in (in)	0.01	2.01 r	
U9/Y (INVX0_RVT)	0.04	2.05 f	
U10/Y (NOR2X0_RVT)	0.09	2.14 r	
pulse_out_reg/D (DFFARX1_RVT)	0.01	2.15 r	
data arrival time		2.15	

clock clk (rise edge)	10.00	10.00	
clock network delay (ideal)	0.00	10.00	
pulse_out_reg/CLK (DFFARX1_RVT)	0.00	10.00 r	
library setup time	-0.06	9.94	
data required time		9.94	

data required time		9.94	
data arrival time		-2.15	

slack (MET)		7.79	

Hold Time Report:

```
dc_shell> report_timing -delay_type min
```

```
*****
```

```
Report : timing
```

```
        -path full
```

```
        -delay min
```

```
        -max_paths 1
```

```
Design : project
```

```
Version: W-2024.09-SP3
```

```
Date   : Sat Aug  8 12:47:37 2026
```

```
*****
```

```
Operating Conditions: tt0p85v25c   Library: saed32rvt_tt0p85v25c
```

```
Wire Load Model Mode: enclosed
```

```
Startpoint: pre_in_reg (rising edge-triggered flip-flop clocked by clk)
```

```
Endpoint: pulse_out_reg
```

```
          (rising edge-triggered flip-flop clocked by clk)
```

```
Path Group: clk
```

```
Path Type: min
```

Des/Clust/Port	Wire Load Model	Library
project	ForQA	saed32rvt_tt0p85v25c

Point	Incr	Path
clock clk (rise edge)	0.00	0.00
clock network delay (ideal)	0.00	0.00
pre_in_reg/CLK (DFFASRX1_RVT)	0.00	0.00 r
pre_in_reg/Q (DFFASRX1_RVT)	0.17	0.17 r
U10/Y (NOR2X0_RVT)	0.08	0.25 f
pulse_out_reg/D (DFFARX1_RVT)	0.01	0.26 f
data arrival time		0.26

clock clk (rise edge)	0.00	0.00
clock network delay (ideal)	0.00	0.00
pulse_out_reg/CLK (DFFARX1_RVT)	0.00	0.00 r
library hold time	-0.01	-0.01
data required time		-0.01

data required time	-0.01
data arrival time	-0.26

slack (MET)	0.27
-------------	------

8. Conclusions and Future Scope

The successful completion of this project demonstrates the end-to-end design, verification, and synthesis of a synchronous rising-edge pulse generator capable of producing a single clock-cycle pulse. Through this internship effort, key learning outcomes were realized across the full digital ASIC implementation pipeline. The development process provided rigorous hands-on experience in drafting edge-detection Register Transfer Level (RTL) logic using Verilog HDL and constructing an automated verification environment utilizing the Universal Verification Methodology (UVM) framework—spanning transaction items, sequences, drivers, monitors, and scoreboard checking. Furthermore, practical expertise was gained in gate-level logic synthesis using Synopsys Design Compiler with the 32nm SAED32 standard cell library, achieving an optimized design footprint of $24.32\ \mu\text{m}^2$ across 7 logic cells with a low total power consumption of $1.598\ \mu\text{W}$. Analyzing Static Timing Analysis (STA) reports deepened comprehension of setup and hold timing constraints, confirming complete timing closure with positive setup slack (+7.79 ns) and hold slack (+0.27 ns) at a 10 ns clock period.

Looking toward practical integration, this single-cycle pulse generator serves as a vital component within digital System-on-Chip (SoC) architectures. Primary applications include synchronizing asynchronous external signals (such as debouncing user push-buttons), triggering single-cycle interrupts for embedded processors, serving as clock-enable pulses for downstream pipeline registers, and driving deterministic state transitions in Finite State Machines (FSMs). The future scope for expanding this project involves modifying the RTL module to support parameterized multi-cycle pulse widths, dual-edge (rising and falling) detection capabilities, and integrated multi-stage synchronizer flip-flops to mitigate metastability risks in cross-clock domain interfaces. Additionally, the UVM verification suite can be upgraded to collect line, toggle, and functional coverage metrics alongside assertion-based verification (ABV) to ensure complete IP maturity for larger industrial VLSI designs.